◆ DRAGONAI · 烛龙智元

AI FDE 工程师就业实训营

**学生讲义**

# Day 3

# 后端开发与 API 系统
# 数据库与模型接入

把 Day 2 的智能客服接成可登录，可持久化，可真实对话的全栈系统

杭州烛龙智元科技有限公司

# 目录


学习目标 1

01 开场 1

1.1 全天课程地图 1

1.2 只有界面的客服原型 2

1.3 原型与系统之间的鸿沟 2

1.4 完整请求链路 2

1.5 核心能力与交付物 3

02 AI 时代的能力构建与项目架构 3

2.1 能力与知识盲区 3

2.2 从前端原型到全栈系统 4

2.3 按瓶颈增强的模块化单体 5

2.4 按业务域拆分的领域服务 6

2.5 多团队交付的平台能力 6

03 前后端如何通过请求协作 7

3.1 请求经过的后端环节 8

3.2 前后端连接的地址与消息 9

3.3 通信方式与选型 9

3.4 HTTP 请求的组成 10

3.5 URL 与 HTTP 方法 11

3.6 Headers 与身份凭证 11

3.7 Body 的数据格式 12

3.8 请求与返回的逐行解读 13

3.9 状态码与错误正文 14

3.10 处理管线与输入映射 14

3.11 Service 与客服消息流程 15

3.12 接口契约与排查证据链 15

3.13 数据库的层级结构 16

3.14 核心表与一对多关系 17

**04 环境配置 18**

4.1 包管理器与运行时的协作 18

4.2 主流后端技术栈 19

4.3 环境自检清单 19

4.4 uv 与 Python 的安装 20

4.5 项目依赖与虚拟环境 20

4.6 MySQL Server 的安装 21

4.7 项目数据库与应用账号 22

4.8 环境问题的排查顺序 22

4.9 环境阶段小验收 22

**05 FastAPI 项目初始化与接口开发 23**

5.1 项目初始化与目录结构 23

5.2 配置与环境变量 24

5.3 Session 生命周期与模型分层 25

5.4 依赖注入与错误结构 25

5.5 接口文档与健康检查 26

5.6 认证与授权 26

5.7 注册与登录接口 26

5.8 JWT 与身份携带 27

5.9 对象级权限与状态码 27

5.10 CORS 与认证模块验收 28

5.11 CRUD 接口清单 28

**06 对话 API 及模型接入与持久化 29**

6.1 发送消息主链路 29

6.2 API Key 与模型 Client 29

6.3 消息角色与模型参数 29

6.4 消息保存与事务边界 30

6.5 流式输出与事件协议 30

6.6 超时重试与限流 31

6.7 输出校验与对话标题 31

6.8 上下文长度与对话验收 31

**07 第二天前端接上第三天后端 32**

7.1 前端改造范围 32

7.2 前端环境变量与 API Client 32

7.3 登录注册页面联调 33

7.4 发送消息的状态机 33

7.5 浏览器里的联调证据 33

7.6 三层测试法 34

7.7 完整演示流程 34

**08 AI/Vibe Coding 常见工程问题与排错 34**

8.1 假后端与假完成 34

8.2 假登录与密钥泄露 35

8.3 编造的依赖与过大的改动 35

8.4 异常处理与同步异步混用 35

8.5 数据库与接口故障 36

8.6 流式故障与重复扣费 36

8.7 提示词的结构 37

8.8 AI 排错所需材料 37

**09 课后作业与评分 38**

9.1 作业要求与真实持久化 38

9.2 身份隔离与接口质量 38

9.3 提高项与提交材料 38

9.4 评分与返工项 39

9.5 理解程度自检 39

**10 环境安装命令附录** 40

10.1 Windows 安装前检查 40

10.2 Windows 安装 uv 与 Python 40

10.3 macOS 与 Linux 安装 uv 41

10.4 Windows 安装 MySQL 41

10.5 macOS 安装 MySQL 42

10.6 课程数据库与连通性验证 42

**练习与作业** 43

复习题 43

课后作业 43

# 学习目标

这一天把 Day 2 做出的智能客服页面接上后端，做成支持登录与数据持久化，并能与真实模型对话的全栈系统，学完后应能做到：

1. 说明前后端，数据库与模型服务各自的职责，并能沿着一次请求从前端函数追到后端路由，再追到 SQL 或模型调用。
2. 区分 HTTP 请求的四个组成部分，读懂响应状态码与错误日志，并能用 curl 或 Swagger 单独验证接口。
3. 完成开发环境的安装，包括 Python 3.12 与 uv，以及 MySQL 与 Navicat，并用版本，路径和连接结果证明环境可用。
4. 在后端项目中指出入口文件与路由，业务服务与数据模型，以及配置与依赖文件的位置。
5. 完成注册登录与会话隔离，让消息持久保存在数据库中，并接入真实模型完成对话。
6. 交付前后端分离的智能客服项目，附上可重复执行的数据库迁移方案，`.env.example` 与 README，以及接口测试证据和演示视频或截图。

# 01 开场

第三天在 Day 2 的前端项目上补齐后端，让一条消息从浏览器发出，经过后端，数据库和模型，再回到页面。

## 1.1 全天课程地图

全天课程分十节，按三个阶段推进：

| 节 | 阶段 | 主要内容 |
| --- | --- | --- |
| 01 到 03 | 基础，架构与请求 | 系统由哪些部分组成，请求怎样在前后端之间流动 |
| 04 到 06 | 环境，接口与对话 | 配置开发环境，开发接口，接入模型并保存对话 |
| 07 到 10 | 联调，排错与验收 | 把前端连接到后端，排查常见工程问题，按作业要求验收 |

十节内容围绕同一个智能客服项目展开。

## 1.2 只有界面的客服原型

Day 2 已经做出智能客服的官网和聊天内页。页面上有输入框与消息气泡，也有侧边栏和登录按钮。学员已经会用 Node 与前端框架，也会用浏览器 F12 和 AI 编程工具。

这个客服只是“看起来完成”。点击“发送”时，页面只是把文字追加到一个数组里，没有发出任何网络请求。界面上出现的回复来自 `setTimeout()` 或预先写好的固定文本，背后没有调用模型。

这一天保留现有的 UI，在当前项目上接入后端。判断客服是否真正接通，先看点击发送后有没有发出网络请求，再看回复是否来自模型。

## 1.3 原型与系统之间的鸿沟

前端原型与可用的全栈系统之间有四道鸿沟：

| 原来的做法 | 这一天要做到 |
| --- | --- |
| 页面状态 | 后端业务逻辑 |
| 内存或浏览器存储 | MySQL 持久化 |
| 前端直连模型 | 后端安全调用模型 |
| 只在本机能运行 | 有接口契约，错误处理和验收证据 |

前三道鸿沟分别关于业务逻辑放在哪里，数据存在哪里，模型由谁调用。第四道鸿沟关于怎样证明功能可用。开发者常说的“我这里能跑”，只说明功能在自己的电脑上能运行，不算完成。功能可用要靠接口契约，错误处理和验收证据来证明。

改造完成后，验收时要拿出三项证据：

- 浏览器 Network 中出现真实请求。
- 后端终端能看到同一请求的日志。
- 刷新页面后数据仍可恢复。

三项证据分别对应前端，后端与数据库三个环节。

## 1.4 完整请求链路

这一天完成后，一条消息的完整链路分五步：

1. 用户在浏览器输入问题。
2. 前端通过 API Client 向后端 API 发送 Token，会话 ID 和问题。
3. 后端完成鉴权，并校验这个会话是否属于当前用户。
4. 后端把用户消息存入 MySQL，再调用模型 API。
5. 后端保存助手回复，再返回给前端，或以流式方式推送给前端。

这条链路覆盖请求与持久化，也覆盖模型调用与历史恢复。与 Day 2 的页面原型相比，链路中的请求，数据和模型调用都是真实的。

### 1.5 核心能力与交付物

这一天的核心能力是看懂系统链路。AI 负责写代码，开发者仍然要能理解“数据从哪里来、到哪里去、失败在哪里”。落到项目上，学员要能在代码里找到入口文件与路由等关键位置。从页面上的一次操作出发，还要能依次找到处理这次操作的前端函数与后端路由，以及最终执行的 SQL 或模型调用。

全班沿 Python 与 FastAPI 这条主线完成同一个全栈项目。Python 3.12 与 uv 负责统一解释器，依赖和可复现环境。FastAPI 用来编写接口并生成 Swagger UI，也负责连接 MySQL 与接入模型。其他主流技术栈只保留选型认知，代码，排错与作业都使用 Python 与 FastAPI。

一天结束时要交付四类成果：

| 交付物 | 内容 |
| --- | --- |
| 项目 | 前后端分离的智能客服项目 |
| 数据库 | MySQL 数据库与可重复执行的迁移方案 |
| 接口 | 注册，登录，用户信息，会话列表，消息发送与读取 |
| 材料 | `.env.example`，README，接口测试证据，演示视频或截图 |

其中的接口测试证据与演示材料，用来证明这条链路确实已经接通。

**小结**

Day 2 的页面只有界面效果，这一天要把业务逻辑，数据存储和模型调用移到后端完成。改造是否完成以证据为准：浏览器能看到真实请求，后端有对应日志，刷新后数据还在。一条消息由前端发起，经后端鉴权与存储后调用模型，结果再回到页面。全天要培养的是读懂整条链路的能力，动手部分统一采用 Python 与 FastAPI。

# 02
# AI 时代的能力构建与项目架构

AI 放大工程能力，项目架构决定产品可以走多远。

### 2.1 能力与知识盲区

Vibe Coding 时代，AI 会放大开发者的能力，也会放大开发者的知识盲区，开发者之间因此出现能力分水岭。按照使用 AI 的方式，程序员大致可以分成五类，前四类对 AI 的依赖程度依次加深：

| 流派 | 做法 | 特点 |
| --- | --- | --- |
| 守旧派 | 不信任 AI 生成的代码，关键领域坚持手写 | 排斥 AI，安全第一，思想传统 |
| 轻度维新派 | 重复性工作交给 AI，复杂和关键代码自己写，自己审查 | 接受 AI，追求稳健，态度谨慎 |
| 半维新派 | 需求和架构清楚后，让 AI 生成大量代码和文档，关键衔接和重要测试自己负责 | 拥抱 AI，追求效率，兼顾谨慎 |
| 维新派 | 清楚业务需求和任务方向，用 prompt 掌控 AI，让 AI 完成 99% 的任务 | 驾驭 AI，追求极致效率，成本较高，基础扎实 |

维新派是激进利用 AI 的高级软件工程师。这类人基础扎实，AI 放大的是已有的能力，所以强者愈强。

第五类人幻想不用学习也能靠 AI 做开发，认为 AI 无所不能，只要会提问并复制粘贴结果，就能完成开发。一个典型场景是：服务器放在远程的家里，既没有做内网穿透，也没有静态 IP，外部无法直接访问。AI 收到问题后，先尝试连通这个 IP，发现连接超时，又去检查网络配置。使用者不懂网络基础，判断不了 AI 的排查方向是否正确。

这种做法的后果是技能萎缩，连 prompt 也写不好。持这种想法的人轻视代码与基础知识，认为程序员无需记忆，AI 可以替代思考。这样的人迟早会在实际工作中碰壁，再回头学习基础知识。

AI 放大的是使用者已有的水平，基础扎实时放大能力，基础薄弱时放大盲区。

## 2.2 从前端原型到全栈系统

项目架构可以分成 L0 到 L5 六级，前三级依次是纯前端原型，最小可用服务和标准全栈系统。图中三栏从左到右依次是 L0，L1 和 L2，每栏左侧是一次请求经过的环节，右侧列出这一级的特点。L0 与 L1 底部的警示框写着这一级的局限，L2 底部的星标写着今天的目标。

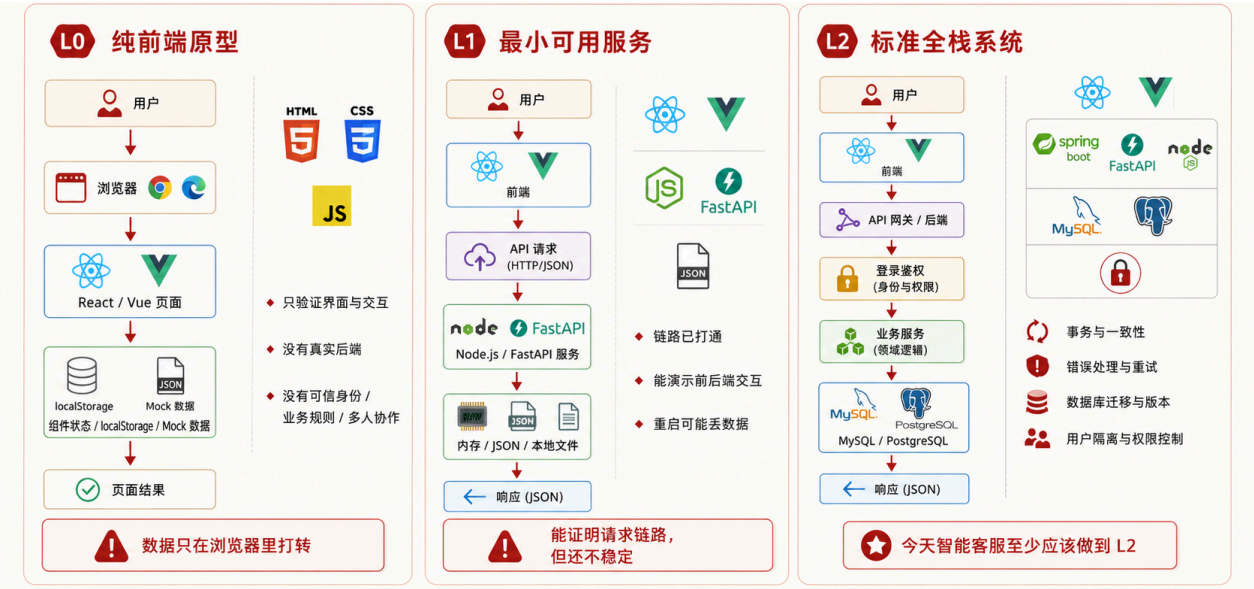



**图 1**　L0 到 L2：纯前端原型，最小可用服务与标准全栈系统。

L0 是纯前端原型。用户在浏览器里操作 React 或 Vue 页面，数据放在组件状态，localStorage 或 Mock 数据里。这一级只能验证界面与交互，数据只保存在浏览器中。由于没有真实后端，这一级也缺少可信身份，业务规则和多人协作。Day 2 的客服页面就处在这一级。

L1 是最小可用服务。前端用 HTTP 发送 JSON 格式的 API 请求，由 Node.js 或 FastAPI 服务处理，响应也以 JSON 返回。数据暂存在内存，JSON 或本地文件里。这一级已经接通请求链路，能演示前后端交互，但数据在重启后可能丢失，系统还不稳定。

L2 是标准全栈系统。请求经过 API 网关或后端，先做登录鉴权，确认身份与权限。业务服务再处理领域逻辑，把数据存进 MySQL 等数据库，最后返回 JSON 响应。这一级还要处理四类工程问题：

- 事务与一致性。
- 错误处理与重试。
- 数据库迁移与版本。
- 用户隔离与权限控制。

今天的智能客服至少要做到 L2。

## 2.3 按瓶颈增强的模块化单体

L3 仍然只有一个核心业务应用，代码按模块组织，称为**模块化单体**。应用内部按职责划分模块，包括用户与鉴权，会话与业务逻辑，以及 AI 调用适配层与后台管理。同一套应用可以用 Docker 容器部署成多个实例，以横向扩展的方式承接更多请求。Web，管理后台和移动端的请求先经过负载均衡，再分发到各个实例。

这一级按真实瓶颈增强系统，遇到哪类问题，就在应用旁边增加对应的组件：

| 遇到的瓶颈 | 增加的组件 |
| --- | --- |
| 接口慢 | 缓存集群 |
| 耗时任务 | 异步消息队列 |
| 文件上传 | 对象存储 |
| 检索复杂 | 搜索服务 |

应用之外还配有任务调度与定时任务，日志与监控，以及备份服务。核心数据库采用主从与多可用区部署。

L3 的每一次增强都由真实出现的瓶颈驱动，没有遇到瓶颈就不加组件。

## 2.4 按业务域拆分的领域服务

业务规模，调用量和团队边界都显著扩大以后，单体应用按业务域拆成多个**领域服务**，就到了 L4。图上方是 Web，App 和 Admin 三类渠道与 API 网关，中间一排是领域服务。再往下依次是异步消息总线，数据存储，以及高可用与容错机制。实线箭头是请求与数据的主要流向，虚线连接网关与各个服务，也连接右侧的服务发现，配置中心和可观测性组件。右端三个方框写着这一级的适用条件，代价和追踪路径。

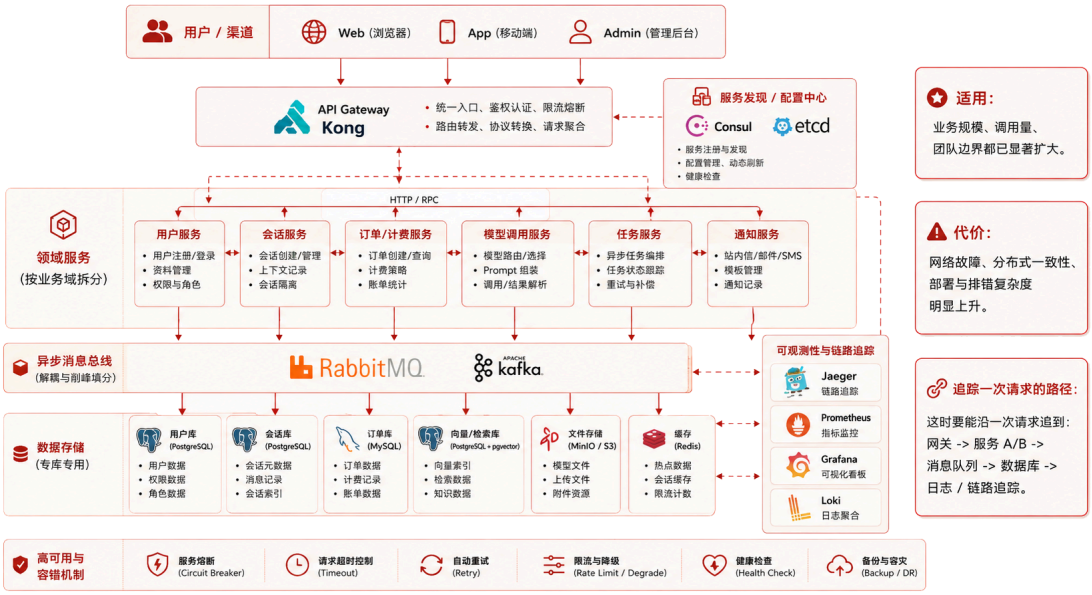



**图 2**　L4 领域服务：按业务域拆分服务，由网关统一入口，配合消息总线，专库专用和链路追踪。

领域服务按业务域划分，例如用户服务，会话服务和模型调用服务，服务之间通过 HTTP 或 RPC 通信。所有请求先进入 API 网关。网关是统一入口，负责鉴权认证与限流熔断，还负责路由转发，协议转换和请求聚合。服务发现与配置中心负责服务注册与发现，配置管理和健康检查。服务之间的异步通信经过消息总线，起到削峰填谷的作用。数据存储专库专用，每类数据使用独立的库，例如用户库，会话库和向量检索库。文件与缓存也单独存放。

拆分的代价是网络故障，分布式一致性，以及部署与排错的复杂度都明显上升。因此这一级配有服务熔断，请求超时控制和自动重试等高可用与容错机制。出了问题，要沿一次请求的路径逐段追踪：从网关到具体服务，再到消息队列和数据库，最后查看日志与链路追踪。

## 2.5 多团队交付的平台能力

L5 面向大型组织，用平台能力支撑多个团队稳定交付。图左侧从上到下是平台的七层，右侧三个方框分别写着适用场景，组织前提，代价与风险。




**图 3**　L5 平台能力：多条业务线的团队通过自助平台，交付流水线和统一规范稳定交付服务。

第一层是多条业务线的团队和平台的内部使用者，这些人通过第二层的平台入口自助查询服务与使用模板，也能自助申请权限和环境。交付流水线从代码托管开始，经过 CI 构建，自动化测试与制品管理，最后完成灰度发布。运行平台负责容器化运行，服务网格和多集群部署。最后三层是可观测与运维，“治理与安全”和多地域容灾。

这一级适合多团队，多业务域协作的大型组织。这类组织对稳定性，可靠性和合规性要求极高，系统还要长期演进。组织上要先具备平台工程团队与能力，统一规范和服务契约要真正落地。代价同样明显：平台的建设与运营成本高，复杂度高，需要持续投入。平台化过度，流程过重，还会拖慢交付。

L5 的核心不是“服务更多”，而是“让大量团队在统一治理下稳定、高效、可审计地交付服务”。

### 小结

AI 放大使用者原有的水平，基础知识仍要开发者自己掌握。项目架构按规模分为六级，今天的智能客服以 L2 标准全栈系统为最低要求，L1 那样的最小可用服务还不稳定。更高的三级代表三种扩展方式：L3 按瓶颈给单体增加组件，L4 按业务域拆分服务，L5 用平台支撑多团队交付。

# 03
# 前后端如何通过请求协作

理解前后端协作，先要分清谁和谁通信，再追踪一次请求怎样变成数据库里的真实记录。

## 3.1 请求经过的后端环节

前端只连接 API，其余环节都由后端统一控制。一次请求从用户的页面出发，在后端依次经过多个环节，每个环节负责一件事。图左边是用户和聊天页面，中间方框里是后端依次经过的处理环节。右上方是模型服务，右下方是数据库。通往这两处的连线上画着锁，表示访问受保护，凭证只在后端。底部一排是日志与追踪，每个环节都用虚线连到这一排，返回给页面的响应也画成虚线。

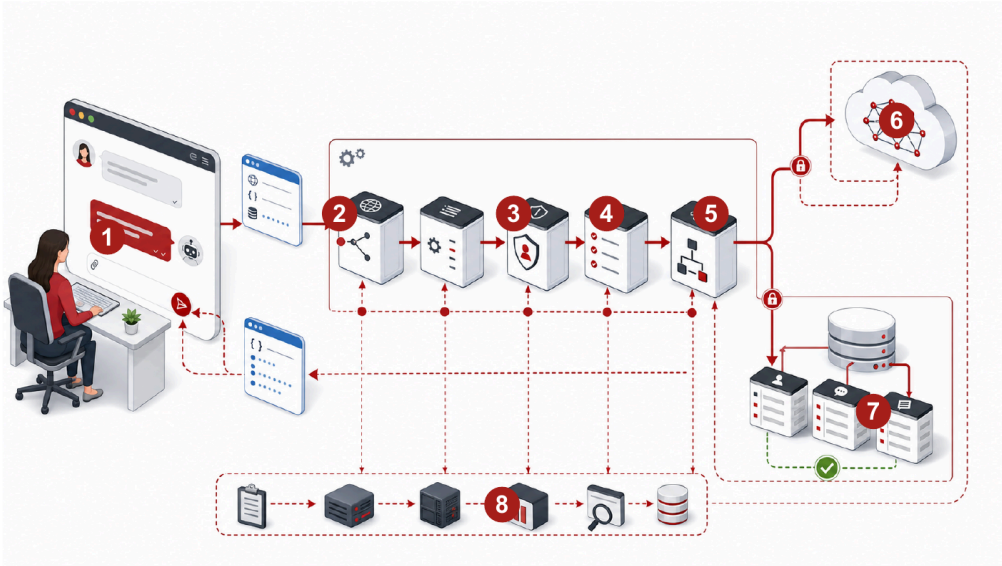



**图 4**　一次请求穿过完整后端系统，图中编号对应八个环节。

图中八个编号的含义是：

| 编号 | 环节 | 负责什么 |
|---|---|---|
| 1 | 前端与 API Client | 收集输入并构造 HTTP 请求 |
| 2 | 路由与中间件 | 定位接口，建立 Request ID 与日志 |
| 3 | 认证与授权 | 确认身份，权限和资源归属 |
| 4 | 参数校验 | 检查类型，必填项和业务边界 |
| 5 | 业务服务 | 组织事务，数据库和外部调用 |
| 6 | 模型 API | 密钥由后端保管，统一超时与重试 |
| 7 | 数据库 | 保存用户，会话，消息与状态 |
| 8 | 日志与追踪 | 沿同一请求定位故障位置 |

日志与追踪贯穿全部环节，出了故障可以沿同一个请求逐段定位。

## 3.2 前后端连接的地址与消息

前后端连接要做两件事：找到对方的地址，再交换消息。后端程序启动后监听一个端口，例如 FastAPI 的 `8000`。客户端按地址找到后端的某个接口，再发送请求，后端处理后返回响应，前端根据响应更新页面。与地址有关的概念有五个：

| 概念 | 含义 |
| --- | --- |
| IP 与域名 | 找到哪一台计算机 |
| 端口 | 找到计算机里的哪个服务进程 |
| 路径 | 找到服务中的哪个业务入口 |
| Origin | 协议，主机，端口的组合 |
| CORS | 浏览器对跨 Origin 请求的安全检查 |

本地开发时，前端常运行在 `localhost:5173`，后端运行在 `localhost:8000`，后者就是本机的后端服务地址。两个地址的端口不同，属于不同的 Origin，所以浏览器会对前端发往后端的请求做 CORS 安全检查。

## 3.3 通信方式与选型

系统之间的通信方式按交互模式分类，分类依据有四条：

- 是否等待响应。
- 是否保持连接。
- 谁能主动发送。
- 是否允许稍后处理。

按这四条依据，常见的通信方式有五种：

| 方式 | 交互模式 | 典型场景 | 初学者判断 |
| --- | --- | --- | --- |
| HTTP 与 HTTPS | 一次请求，一次响应 | 登录，CRUD，文件上传，模型调用 | 业务 API 的默认选择 |
| SSE | 服务器持续推送单向事件 | 大模型逐字输出，任务进度 | 浏览器接收流式文本很方便 |
| WebSocket | 长连接，双向实时通信 | 在线协作，游戏，即时消息 | 双方都要随时主动发消息 |
| RPC 与 gRPC | 像调用函数一样调用远端服务 | 内部服务间高性能通信 | 服务端工程常见，浏览器主线较少 |
| 消息队列 | 异步投递，稍后处理 | 邮件，报表，耗时任务，削峰 | 发送方无需等待立即完成 |

表中除 RPC 以外的四种方式，消息节奏各不相同。图中每一行是一种方式，双向箭头表示双方都能发送，单向箭头表示消息只朝一个方向流动。

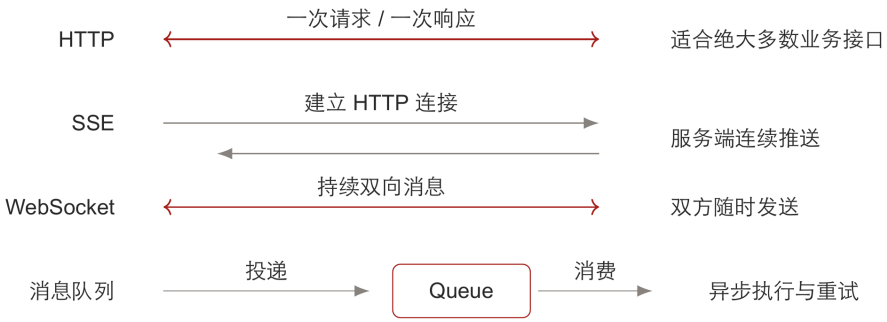



**图 5**　四种通信方式的消息节奏对比。

HTTP 的一次请求对应一次响应，适合绝大多数业务接口。SSE 先由客户端建立 HTTP 连接，之后服务端沿这条连接连续推送。WebSocket 建立后形成持续的双向通道，双方随时可以发送消息。消息队列由发送方把消息投递到 Queue，消费方再取出处理，适合异步执行与重试。

选型时，CRUD 用 HTTP，模型流式输出用 SSE，强实时双向交互用 WebSocket。

## 3.4　HTTP 请求的组成

后端开发先从 HTTP 的请求与响应学起。前端页面发出 HTTP 请求，HTTP API 完成业务处理，得到的业务结果再作为响应返回前端页面。HTTP 适合作为起点，有四个特点：

- 各种客户端都能调用，包括浏览器与移动端，也包括脚本和服务器。
- 每次调用都有明确的输入，输出和状态码。
- curl 与 Swagger 等工具都能脱离页面独立验证。
- HTTPS 在 HTTP 之外增加了加密与服务端身份验证。

学习 HTTP，要能看懂一个请求的来源与目的，以及请求内容与返回结果。一个 HTTP 请求由四部分组成，四部分共同表达这次调用的意图：

| 部分 | 表达什么 |
| --- | --- |
| 方法 Method | 想查询，创建，修改还是删除 |
| URL | 目标服务，资源路径与查询条件 |
| Headers | 内容类型，身份，来源，缓存与追踪元信息 |
| Body | 真正提交的业务数据或文件内容 |

四部分里，方法与 URL 说明要对哪个资源做什么，Headers 与 Body 提供完成这件事所需的上下文与数据。

## 3.5 URL 与 HTTP 方法

URL 负责把请求准确送到某个资源。以一个消息接口的地址为例：

```
https://api.example.com:443/api/v1/conversations/42/messages?limit=20
```

这个地址由五个部分组成：

| 组成部分 | 示例中的值 |
| --- | --- |
| 协议 | `https` |
| 主机 | `api.example.com` |
| 端口 | `443` |
| 路径 | `/api/v1/conversations/42/messages` |
| 查询参数 | `limit=20` |

资源 ID 放在 Path 里，路径中的 `42` 就是会话 ID。筛选与分页条件放在 Query 里，例如 `limit=20`。

HTTP 方法表达对资源的操作语义：

| 方法 | 常见意图 | 智能客服示例 | 重复调用要点 |
| --- | --- | --- | --- |
| GET | 查询资源 | 获取会话与消息列表 | 应保持只读 |
| POST | 创建资源或执行业务动作 | 创建会话，发送消息 | 可能重复创建或扣费 |
| PUT | 整体替换 | 替换完整用户配置 | 相同请求结果应一致 |
| PATCH | 局部修改 | 修改会话标题或状态 | 只更新明确字段 |
| DELETE | 删除或标记删除 | 删除会话 | 重复删除需定义结果 |

表中最后一列说明重复调用时的要求。网络重试可能让同一个请求不止一次到达后端。GET 应保持只读，重复调用不改变数据。POST 每执行一次都可能新建一条记录，重复调用可能造成重复创建或重复扣费。

同一个请求执行一次与重复执行多次，对数据产生的效果相同，这种性质称为**幂等**。对可能重试的创建与付费动作，要设计 Idempotency-Key 或业务去重键，让重复到达的请求只生效一次。

## 3.6 Headers 与身份凭证

Headers 保存请求的上下文与控制信息，适合存放元信息。常用的 Header 有七个：

| Header | 存放什么 | 作用与注意事项 |
| --- | --- | --- |
| Content-Type | Body 的格式 | 常见值 `application/json`，`multipart/form-data` |
| Accept | 客户端希望接收的格式 | 常见为 JSON 或事件流 |
| Authorization | Bearer Token 等凭证 | 后端据此认证身份，日志需脱敏 |
| Cookie | Session ID 等浏览器状态 | 浏览器自动携带，需配置 SameSite 与 Secure |
| Origin | 页面来源 | 后端 CORS 策略据此判断是否允许 |
| X-Request-ID | 一次请求的追踪 ID | 关联网关，后端，数据库与日志中的记录 |
| Idempotency-Key | 业务去重标识 | 防止网络重试造成重复创建或重复扣费 |

密码，Token 与 API Key 都不能写进 URL，凭证要放在 Header 里传递。

身份凭证随请求进入后端，常见的方式有三种：

| 方式 | 请求放在哪里 | 后端怎样确认 | 常见场景 |
| --- | --- | --- | --- |
| Bearer JWT | Authorization Header | 验签并检查过期时间，签发方与受众 | 前后端分离 API，移动端 |
| Cookie Session | Cookie Header | 用 Session ID 查询服务端会话 | 同站 Web 应用，传统后台 |
| API Key | 自定义 Header | 查询调用方，额度与权限范围 | 服务间调用，模型供应商 API |

认证回答“你是谁”，授权继续判断“你能否访问这条会话或消息”。身份凭证进入请求后，后端仍要继续授权：先从可信凭证中得到当前用户 ID，再把这个 ID 加入数据库查询条件，按当前身份校验资源归属。

通过认证只说明后端知道了调用者是谁，访问每一条会话与消息之前，还要确认这条数据属于当前用户。

## 3.7 Body 的数据格式

Body 根据数据形态选择格式，Content-Type 告诉后端 Body 采用的是哪种格式。

| 格式 | Content-Type | 适用数据 |
| --- | --- | --- |
| JSON | `application/json` | 对象，数组，字符串，数字，布尔值，是主流业务 API 的常用格式 |
| 表单 | `application/x-www-form-urlencoded` | 简单键值对，兼容传统表单 |
| 文件表单 | `multipart/form-data` | 图片，PDF 与同时提交的文字字段 |
| 二进制 | `application/octet-stream` | 单个原始文件流或协议数据 |
| 无 Body | 无需 Content-Type | GET 查询，204 响应等没有正文的场景 |

后端收到 Body 后要逐项校验：格式与字段类型，必填项与长度，文件大小与 MIME 类型。

主流业务 API 用 JSON，带文件的提交用文件表单，没有正文的请求不需要 Content-Type。

## 3.8 请求与返回的逐行解读

把一条发送客服消息的真实请求逐行读懂：

```
POST /api/v1/conversations/42/messages HTTP/1.1
Host: localhost:8000
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
X-Request-ID: req_20260713_001
Idempotency-Key: msg_client_8848

{"content":"如何申请退款？"}
```

第一行是请求行：POST 表示创建消息，路径中的 `42` 是会话 ID，`HTTP/1.1` 是协议版本。随后五行是 Headers，各自的作用是：

| Header | 在这条请求中的作用 |
| --- | --- |
| `Host` | 指向本机的后端服务 |
| `Content-Type` | 说明 Body 是 JSON |
| `Authorization` | 用 Bearer Token 证明调用者身份 |
| `X-Request-ID` | 跨系统查日志时使用 |
| `Idempotency-Key` | 防止重复发送 |

Headers 结束后空一行，最后是 JSON Body，保存用户的问题。

后端处理成功后返回的响应是：

```
HTTP/1.1 201 Created
Content-Type: application/json
X-Request-ID: req_20260713_001

{
  "id": 991,
  "role": "user",
  "content": "如何申请退款？",
  "created_at": "2026-07-13T15:30:00Z"
}
```

响应由三部分组成。第一行是状态行，包含协议版本，状态码与简短描述，`201 Created` 表示消息已经创建。Headers 描述响应格式与缓存，以及跨域和追踪信息，同样在结束后空一行。Body 存放成功数据或结构化错误，这里返回的是新建消息的各个字段。

前端先判断状态码，再解析对应的正文。请求与响应带着同一个 `X-Request-ID`，后端日志也记录这个 ID，联调时凭这个 ID 就能把前端看到的结果与后端日志关联起来。

## 3.9 状态码与错误正文

状态码把结果分成可行动的类别，前端与调用方按类别决定下一步动作：

| 类别 | 代表状态码 | 前端与调用方动作 |
| --- | --- | --- |
| 成功 | 200，201，204 | 读取数据，更新页面，204 不读取正文 |
| 请求问题 | 400，409，422 | 修正字段，处理重复冲突，展示校验信息 |
| 身份权限 | 401，403，404 | 重新登录，提示无权限，或隐藏资源存在性 |
| 频率限制 | 429 | 读取 Retry-After，降低频率并执行有限重试 |
| 服务异常 | 500，502，503 | 保存 Request ID，查看后端日志，按策略重试或降级 |

出错时，错误正文统一返回四个字段：错误码 `code` 与提示信息 `message`，以及详细信息 `details` 和请求编号 `request_id`。这样既方便前端展示，也方便工程排查。

前端拿到响应，先看状态码属于哪一类，再决定如何处理。

## 3.10 处理管线与输入映射

后端按固定管线处理一次请求。收到请求后，先做路由匹配，再经过中间件，然后依次完成认证授权与参数校验，最后进入业务服务。业务服务通过 Repository 或 ORM 读写数据库，需要时再调用模型或外部 API。处理结果逐层转换为 JSON，状态码与响应 Headers，返回给调用方。

请求数据分散在五个位置，后端要先把这些数据映射成可校验的输入：

| 来源 | 示例 | 后端处理 |
| --- | --- | --- |
| Path | `/conversations/42` | 转成 `conversation_id`，并检查资源是否存在与归属 |
| Query | `limit=20, cursor=...` | 校验范围，默认值，排序与分页规则 |
| Headers | Authorization 与 Request ID | 建立当前身份，追踪上下文与幂等上下文 |
| Body | JSON 中的 `content` 与 `title` | 用 Schema 或 DTO 校验类型，长度，枚举和必填项 |
| File | multipart 上传的图片或 PDF | 校验 MIME，大小，扩展名，恶意内容和权限 |

字段有问题时，请求在进入业务服务之前就返回 400 或 422。参数校验位于业务服务之前，业务代码收到的都是校验过的输入。

## 3.11 Service 与客服消息流程

已经通过鉴权与校验的请求上下文交给 Service。Service 对应一个业务用例，例如注册用户，发送消息或创建订单。Service 负责组织一次完整的业务处理，产出的业务结果以响应或事件的形式交出。图中 Service 下方是四类协作对象，处理过程中按业务需要调用。

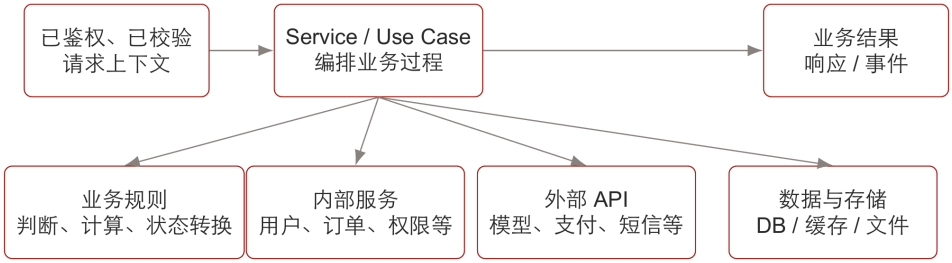



**图 6**　Service 按业务需要协调四类对象，完成一次业务处理。

业务规则负责判断，计算和状态转换。内部服务指用户，订单，权限等模块。外部 API 包括模型，支付，短信等。数据与存储包括数据库，缓存和文件。调用外部 API 时，Service 要处理鉴权与超时，重试与限流，还要做到幂等。读写数据库通过 ORM，Repository 或参数化 SQL 完成，也可以使用缓存和对象存储。

以智能客服的发送消息为例，一条消息的完整流程分九步。图中第一排从前端发送开始，到保存助手消息为止。第二排从右向左依次是返回结果，前端更新页面，刷新后重新查询。

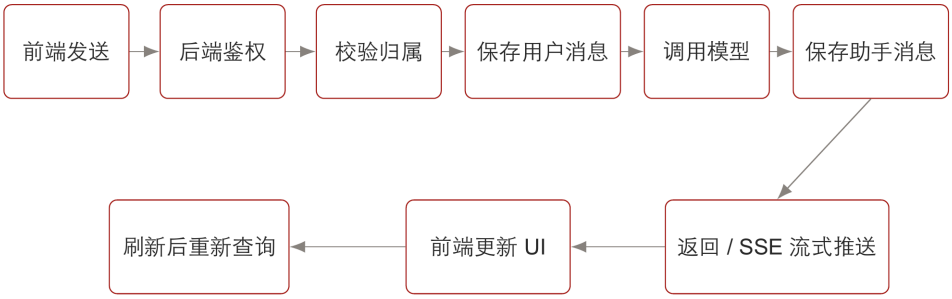



**图 7**　一条客服消息的完整业务流程。

顺序上有两处安排。用户消息先写入数据库，再调用模型，模型调用失败时仍能看到真实的请求历史。助手回复完成后再写入数据库，刷新页面时从数据库恢复完整会话。

验收时，刷新页面并重启后端后仍能恢复消息，才能证明后端与数据库确实已经接通。

## 3.12 接口契约与排查证据链

前端，后端与各种工具通过接口契约保持一致。OpenAPI 描述接口的全部约定，包括请求的路径，方法与参数，Body 的 Schema，以及响应与鉴权要求。前端按契约构造请求，后端按契约验证并返回。JSON 字段的名称，类型与必填规则要保持稳定，日期格式和错误结构也一样。

有了契约，可以先用工具单独验证后端，再接页面，出了故障更容易分清问题出在哪一边。常用工具有四种：

| 工具 | 主要用途 |
| --- | --- |
| Swagger UI | 查看 Schema，快速调试接口 |
| Apifox | 设计，Mock，调试，环境与自动化 |
| Postman | Collection，环境，脚本与协作 |
| curl | 最小复现，README，服务器与 CI |

在工具里共享接口时只共享示例，真实密钥留在受控环境里。

前后端出了问题，沿一条证据链逐段定位：

| 顺序 | 查看位置 | 检查内容 |
| --- | --- | --- |
| 1 | 浏览器 Network | 请求是否发出 |
| 2 | curl 或 Swagger | 能否独立复现 |
| 3 | 后端日志 | 路由，参数，异常 |
| 4 | 数据库查询 | 记录是否存在 |

每一段都要留下证据。浏览器里保存请求的 URL，方法与状态码，以及请求和响应的 Body。后端日志用同一个 Request ID 检索。最终的数据状态用 SQL 客户端确认。要让这条链路在本机运行起来，还需要先安装开发环境，包括 Python 与 uv，以及 MySQL 和数据库客户端。

请求连接前端与后端，后端把输入变成可信的业务结果与持久化数据。

## 3.13 数据库的层级结构

数据库把业务数据组织成可查询的结构。从 Server 到行与列，一共四个层级：

| 层级 | 智能客服中的例子 |
| --- | --- |
| MySQL Server | 监听 3306 端口 |
| Database | `customer_service` |
| Table | `users`，`conversations`，`messages` |
| Row 与 Column | 一条记录与记录里的字段 |

表结构规定每个字段的名称与类型，以及是否必填和默认值，每一行是一条业务记录。**主键**用来定位一行，同一张表里每行的主键值各不相同。**外键**表达表与表之间的归属关系。索引加快常用查询。事务确保一组写入同时成功或同时回滚。增删改查都通过 SQL 完成，后端通常借助 ORM 生成并执行 SQL。

四个层级逐层包含：Server 管理多个数据库，每个数据库包含多张表，每张表由行和列组成。

## 3.14 核心表与一对多关系

智能客服用三张核心表分别保存身份，会话与消息：

| 表 | 主要字段 | 保存什么 |
| --- | --- | --- |
| users | id，email，password_hash | 用户身份 |
| conversations | id，user_id，title | 会话 |
| messages | id，conversation_id，role，content | 消息 |

一个用户可以有多个会话，一个会话可以有多条消息，两层都是一对多关系，写作 1:N。conversations 表的 user_id 指向所属用户，messages 表的 conversation_id 指向所属会话，每条数据都能找到所属的用户与上下文。

常见操作对应的数据库动作是：

| 操作 | 数据库动作 |
| --- | --- |
| 注册 | 向 users 新增一条记录 |
| 登录 | 按邮箱查出用户，再用 password_hash 校验密码 |
| 创建会话 | 向 conversations 新增记录，并写入当前用户 ID |
| 发送消息 | 先写入 user 消息，模型返回后再写入 assistant 消息 |
| 查询历史 | 按 conversation_id 取出这个会话的消息，按先后顺序读取 |

重启后端后，这些记录仍然保存在数据库中。增删改查合称 CRUD，对应 Create 新增与 Read 查询，以及 Update 修改与 Delete 删除。

注册登录，会话隔离和对话历史都建立在这三张表和两层一对多关系之上。

---

**小结**

一次请求先按地址找到后端，再按 HTTP 的约定交换消息，业务接口默认使用 HTTP，模型的流式输出使用 SSE。读懂请求要分清四个组成部分，读懂响应要先看状态码的类别。后端按固定管线处理请求，认证之后还要授权，参数校验之后才进入 Service。消息最终保存在三张一对多关联的表里，刷新页面或重启后端都不会丢失。排查故障时沿证据链逐段定位，从浏览器一直查到数据库。

# 04
# 环境配置

环境配置要在本机完成 Python 与 uv，以及 MySQL 与 Navicat 的安装与配置，并逐项拿出验证证据。各系统可直接执行的安装命令收在第 10 节环境安装命令附录。

## 4.1 包管理器与运行时的协作

后端项目从源码变成运行中的服务，需要包管理器与运行时协作：**包管理器**准备依赖，**运行时**执行代码与依赖。图中六个编号标出参与协作的各个角色，红色箭头表示依赖与代码的流向。

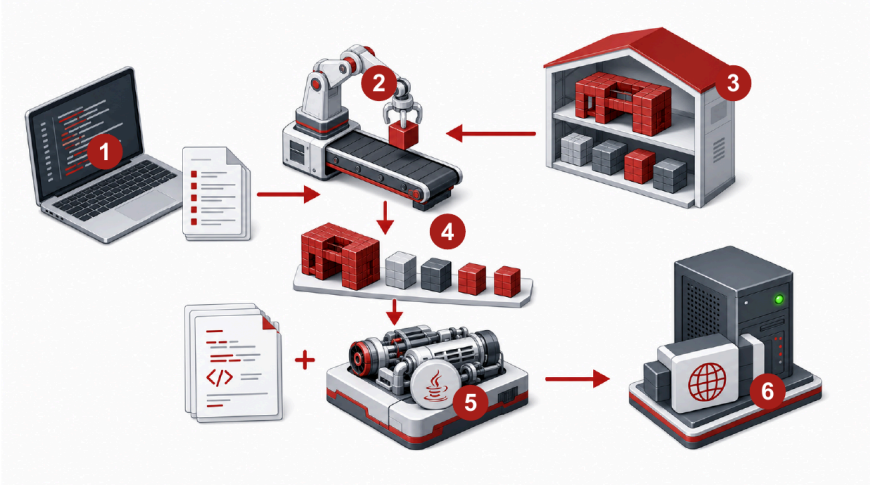



**图 8**　包管理器从包仓库取来框架与依赖包，运行时把代码与依赖一起运行成后端服务。

六个编号的含义是：

| 编号 | 角色 | 作用 |
|---|---|---|
| 1 | 项目源码与依赖清单 | 开发者声明项目需要哪些包和版本 |
| 2 | 包管理器 | 读取清单，解析版本，下载并锁定依赖 |
| 3 | 包仓库 | 集中保存可下载的框架与第三方库 |
| 4 | 框架与依赖包 | 框架提供项目骨架，库提供具体能力 |
| 5 | 运行时 | 加载，解释或编译代码与依赖，Java 项目用 JDK |
| 6 | 运行中的后端 | 启动端口，开始接收 HTTP 请求 |

开发者先在依赖清单里声明需要的包。包管理器读取清单，从包仓库下载框架与依赖包，并锁定版本。运行时再把源码和依赖一起加载执行，后端启动端口后开始接收 HTTP 请求。课程的 Python 主线里，包管理器是 uv，运行时是 Python 解释器。

包管理器解决依赖从哪里来，运行时负责把代码运行起来。

## 4.2 主流后端技术栈

主流后端技术栈各有优势。课程使用 Python 与 FastAPI，先掌握各技术栈共同的后端概念。

| 维度 | Python 与 FastAPI | TypeScript 与 NestJS | Java 与 Spring Boot |
| --- | --- | --- | --- |
| 运行与依赖 | Python，uv | Node.js，pnpm | JDK，Maven 或 Gradle |
| 优势 | AI 相关的库与工具丰富，开发验证快 | 前后端语言统一，模块化清晰 | 企业级框架与工具成熟，长期维护与管控能力强 |
| 典型场景 | AI 应用，数据 API，快速产品验证 | SaaS，BFF，实时 Web 服务 | ERP，金融，政企核心业务 |
| 常见代价 | 大项目要主动约束类型与目录 | 要理解异步 I/O 与 npm 的包体系 | 概念较多，启动和构建更重 |
| 课程安排 | 完整安装，编码，调试与作业 | 了解选型场景 | 了解选型场景 |

另外两种技术栈只需了解适用场景，动手部分全部使用 Python 与 FastAPI。课程主线由 Python 3.12 与 uv，以及 FastAPI 与 MySQL 组成。

三种技术栈的运行时，包管理器和框架各不相同，后端概念是共同的。

## 4.3 环境自检清单

环境配置完成后，用这组命令逐项自检：

```
git --version
uv --version
uv run python --version
mysql --version
mysql -u root -p
SELECT VERSION();
```

每条命令确认的内容是：

| 命令 | 确认什么 |
| --- | --- |
| `git --version` | 代码版本管理工具可用 |
| `uv --version` | Python 项目与包管理器可用 |
| `uv run python --version` | 项目实际使用 Python 3.12 |
| `mysql --version` | MySQL 客户端存在 |
| `mysql -u root -p` | 能连接 MySQL Server |
| `SELECT VERSION();` | 登录后能执行查询，Server 工作正常 |

`mysql --version` 只说明客户端存在，Server 是否正常要通过后两条命令确认。最后用 Navicat 建立本地连接，看到数据库并执行 SQL。

自检合格的标准是版本正确，路径明确，数据库能连接也能查询。

## 4.4 uv 与 Python 的安装

课程选 Python 作为主线，原因是语法简洁，FastAPI 能自动生成接口文档，与 AI/RAG 相关的库与工具衔接自然。Python 解释器与第三方包是两回事，安装了 Python 不等于已经安装了 FastAPI。项目统一使用 Python 3.12，兼顾兼容性与可复现性。

uv 是 Python 的项目与包管理器，安装方式按系统区分：

| 系统 | 安装方式 |
| --- | --- |
| macOS/Linux | uv 官方安装脚本，或 `brew install uv` |
| Windows PowerShell | uv 官方 PowerShell 安装命令，或<br>`winget install --id=astral-sh.uv -e` |

安装后重开终端，执行 `uv --version`。找不到命令时，先查 PATH，再决定是否重装。

uv 可用以后，执行 `uv python install 3.12`，安装指定的 Python 3.12。电脑上的系统 Python，项目 Python 和 IDE 里选的解释器可能不是同一个，选错解释器是常见问题。项目实际用的是哪一个，以项目目录中 `uv run python --version` 的输出为准。

## 4.5 项目依赖与虚拟环境

创建后端项目，并在项目里固定 Python 版本：

```
uv init backend-python --python 3.12
uv python pin 3.12
uv run python --version
```

第一条命令创建名为 `backend-python` 的练习项目，用来验证环境。进入这个目录后执行后两条：一条把版本约束写进项目，一条确认当前项目实际使用的解释器。正式的智能客服后端项目 `customer-service-backend` 在第 05 节另行创建。

项目需要的依赖用 `uv add` 添加：

```
uv add 'fastapi[standard]'
uv add sqlmodel alembic pymysql
uv add pydantic-settings
uv add 'pwdlib[argon2]' pyjwt
uv sync
uv tree
```

前三条添加 FastAPI，数据库与配置相关的依赖，第四条添加认证相关的依赖。认证与模型相关的依赖还包括 `python-multipart` 和所用模型的 SDK。

依赖信息记在两个文件里。`pyproject.toml` 是依赖清单，记录项目名称，Python 版本范围和直接依赖，说明项目需要什么。`uv.lock` 是锁文件，锁定直接与间接依赖的精确版本，确保不同电脑上的安装结果一致。

日常维护依赖用四个命令：

| 命令 | 作用 |
| --- | --- |
| `uv add` | 增加依赖 |
| `uv remove` | 删除依赖 |
| `uv sync` | 按锁文件同步环境 |
| `uv tree` | 查看依赖树 |

报错时，先确认当前目录，Python 版本和实际使用的虚拟环境。

**虚拟环境**让每个项目拥有独立的依赖，避免全局安装互相污染。uv 会自动管理项目里的 `.venv` 目录，用 `uv run ...` 执行命令时不依赖手动激活环境。

提交 Git 时，`pyproject.toml` 与 `uv.lock` 都要提交。`.venv` 体积大且可以重建，应放进 `.gitignore`。`.env` 同样要忽略，不进入提交记录。

两个依赖文件一起提交，另一台电脑才能安装出完全相同的环境。

## 4.6 MySQL Server 的安装

课程使用 MySQL 8.4 LTS，生产数据库的部署放在 Day 5 展开。安装时要完成服务配置，并记录连接参数。

Windows 上，从 MySQL 官方渠道安装 8.4 LTS 64 位版本，用 Configurator 配置服务。安装后记录 root 密码与端口 `3306`，以及 Windows 服务名和字符集。

macOS 上可以用官方的 DMG/PKG 安装包，也可以用 Homebrew。选定一种安装路线后，就使用这条路线对应的服务管理命令。学员要学会启动，停止和查看服务状态，并记录 root 密码和端口。

安装完成后，按自检清单做一次连接查询，确认 Server 正常。命令行客户端的基本连接和执行 SQL 必须会用，便于在服务器环境中排错。Navicat 用来保存连接与浏览表结构，也能查看数据和执行查询，排错仍然依靠命令行。

图形工具负责日常查看，命令行负责排错，两种方式都要掌握。

## 4.7 项目数据库与应用账号

应用账号与 root 必须分开。root 用于数据库管理，项目单独创建一个应用账号，例如 `customer_app`。这个账号只授予项目数据库所需的权限，来源限制为本机开发环境。数据库账号的密码放在环境变量里，不写进源码，也不出现在提交记录中。

创建项目数据库与应用账号分三步，对应的 SQL 命令收在第 10 节环境安装命令附录：

1. 创建 `customer_service` 数据库，字符集使用 `utf8mb4`。
2. 创建本地应用用户，只授予这个数据库的必要权限。
3. 用应用账号重新连接，执行读写测试，证明权限配置有效。

应用平时只用这个账号连接数据库，root 不出现在应用的配置里。

## 4.8 环境问题的排查顺序

环境配置出问题时，同一类故障沿固定顺序收集证据，避免让 AI 随意尝试命令。五类常见现象的检查项与证据是：

| 现象 | 先检查 | 常用证据 |
| --- | --- | --- |
| 命令不存在 | 安装是否完成，终端是否重开，PATH | `which`，`where`，`Get-Command` |
| 版本不对 | Python 是否为 3.12，IDE 使用的解释器是否与项目同源 | `uv run python --version`，实际路径 |
| 包不存在 | 当前目录，锁文件，包管理器，虚拟环境 | `uv tree`，`pyproject.toml`，`uv.lock` |
| 端口占用 | 占用进程与服务端口配置 | `lsof`，`netstat`，后端启动日志 |
| 网络失败 | URL，代理，证书，镜像与防火墙 | `curl -v`，状态码，DNS |

排查的底线是：先保留第一段错误，完整命令和环境信息，再向 AI 提问。

## 4.9 环境阶段小验收

环境阶段结束时做一次小验收，验收项有三条：

1. FastAPI 项目有一个可访问的 `/health`。

2. 学员能说出启动命令与停止方式，以及服务端口和入口文件。

3. MySQL 服务可连接，Navicat 能查看 `customer_service`，FastAPI 能读取 `DATABASE_URL`。

验收时要拿出三项证据：运行时与包管理器的版本，实际执行文件的路径，`/health` 启动后在对应端口可以访问。每项检查都要留存记录，可以是命令与版本输出，也可以是连接结果或截图。

---

**小结**

包管理器与运行时分工合作，课程中分别由 uv 和 Python 3.12 承担。环境是否可用要用证据说明，版本，路径和数据库连接都要逐项核对。依赖清单与锁文件一起提交 Git，虚拟环境目录与环境变量文件不提交。数据库选用 MySQL 8.4 LTS，项目只通过权限受限的独立账号读写自己的数据库。遇到环境故障，先按现象收集证据，再带着证据向 AI 提问。

# 05
# FastAPI 项目初始化与接口开发

后端开发的顺序是初始化项目，用 Swagger 验证接口，再完成登录注册与会话 CRUD。

## 5.1 项目初始化与目录结构

后端项目的第一个目标是得到一个可启动，可访问，可调试的最小工程，并完成第一个请求。初始化按五个步骤进行：

1. 创建项目并固定 Python 3.12。

2. 安装项目所需的依赖。

3. 创建 `app/main.py`，实现 `GET /health`。

4. 启动开发服务器，观察端口与自动重载日志。

5. 用浏览器，Swagger UI 和 curl 三种方式验证。

对应的初始化与启动命令依次是：

```
uv init customer-service-backend --python 3.12
cd customer-service-backend
uv add 'fastapi[standard]' sqlmodel alembic pymysql
uv add pydantic-settings 'pwdlib[argon2]' pyjwt
uv run fastapi dev app/main.py
curl http://localhost:8000/health
```

前两行创建项目，固定 Python 3.12，并进入项目目录，中间两行安装依赖，第五行启动开发服务器，最后一行用 curl 请求健康检查接口。两条 `uv add` 装入的依赖各有分工：

| 依赖 | 用途 |
| --- | --- |
| `fastapi[standard]` | FastAPI 框架与开发服务器 |
| `sqlmodel` | ORM |
| `pymysql` | MySQL 驱动 |
| `alembic` | 数据库迁移 |
| `pydantic-settings` | 读取配置 |
| `pwdlib[argon2]` | 用 Argon2 处理密码 |
| `pyjwt` | 签发与校验 JWT |

数据库迁移把表结构的每一次变化记录成可以重复执行的脚本，新环境按顺序执行迁移，就能建出同样的表结构。服务启动后有三个入口：`http://localhost:8000/docs` 是 Swagger UI 接口文档，`/openapi.json` 是 OpenAPI 描述文件，`/health` 是健康检查接口。

项目目录按职责分层，推荐的结构中，每个目录的职责写在右侧注释里：

```
app/
  main.py          # 入口
  api/             # Router
  services/        # 业务流程
  repositories/    # 数据访问
  models/          # ORM
  schemas/         # 输入输出
  core/            # 配置与安全
  db/              # Session / migration
  clients/         # 模型供应商
alembic/           # 迁移脚本
tests/             # 测试
.env.example
README.md
```

其中 `core/config.py` 集中管理配置，`schemas/` 放输入输出模型，`clients/` 封装对模型供应商的调用。这一阶段的验收标准是三类信息都能实际查证：命令与版本，服务端口，实际执行文件的位置。

## 5.2 配置与环境变量

连接地址，密钥与模型设置都通过环境变量配置，常用字段有五个：

| 字段 | 含义 |
| --- | --- |
| DATABASE_URL | 数据库连接 URL |
| MODEL_API_KEY | 模型 API Key |
| MODEL_BASE_URL | 模型服务地址 |
| MODEL_NAME | 模型名 |
| JWT_SECRET | 签发 JWT 的密钥 |

本地的 `.env` 文件不提交，`.env.example` 只写字段名和安全的示例值。应用启动时校验必需的配置，缺少某一项就立即给出清晰的错误。

`DATABASE_URL` 把连接数据库所需的全部信息写进一个字符串，连接 MySQL 时各部分的位置是：

```
mysql+pymysql://用户名:密码@主机:端口/数据库名?charset=utf8mb4
```

开头的 `mysql+pymysql` 指明数据库类型与驱动，末尾的 `charset` 指定字符集。URL 用 `:`，`@` 和 `/` 分隔各部分，密码里含有这些字符时必须先做 URL 编码，否则连接串会被拆错，表现为“明明密码正确却连接失败”。本地数据库的主机通常写 `localhost`，Docker 环境中写 Compose 服务名，这部分在 Day 5 动手操作。

## 5.3 Session 生命周期与模型分层

数据库 Session 的生命周期与请求一致。每个请求获取一个独立的 Session，用完后提交或回滚，再关闭。不能把一个全局 Session 永久复用给所有请求。异常发生时要确保回滚，避免连接停留在错误的事务状态。

输入 Schema，数据库 Model 和输出 Schema 职责不同，不应完全混用，注册接口是典型的例子。注册输入包含明文密码，数据库只保存 `password_hash`，返回给前端的用户输出也不能带上 `password_hash`。可写字段由服务端控制，避免客户端上传 `is_admin: true` 之类的越权字段。

## 5.4 依赖注入与错误结构

**依赖注入**的做法是：路由只声明需要的对象，例如数据库 Session，当前用户或配置，由框架负责提供。这样认证逻辑可以集中复用，不必在每个接口里复制 Token 解析代码，测试时也便于替换数据库或模型 Client。

接口的响应与错误结构要统一。成功响应保持稳定的字段，错误响应至少包含机器可读的 `code` 和人类可读的 `message`。异常堆栈与 SQL 属于内部细节，密钥与供应商响应属于敏感信息，两类内容都不直接发给浏览器。前端依据稳定的 HTTP 状态码和机器可读的错误码处理异常。

## 5.5 接口文档与健康检查

FastAPI 根据路由和 Schema 自动生成 OpenAPI 描述 `/openapi.json`，并在 `/docs` 用 Swagger UI 展示。路由和 Schema 一旦修改，文档随之更新，所以 OpenAPI 与 Swagger 被称为“活的接口说明书”。在 Swagger 中可以走完整条流程：注册并获取 Token，用 Token 完成授权，再创建会话并发送消息。文档能打开不代表业务正确，还要逐项校验状态码，数据库记录和权限。

健康检查与**就绪检查**的分工是：`/health` 表明进程活着，在此基础上增加数据库探测，可以确认依赖可用。健康检查中不调用昂贵的大模型 API。Day 5 的部署和监控会使用这些端点，所以端点的命名与响应在今天先规范好。

日志采用结构化格式，每条记录包含三组字段：时间与级别，请求 ID 与路径，以及耗时，状态码和错误类别。凭据与敏感内容不打印到日志，凭据指密码，完整的 Token 和 API Key，敏感内容指用户的聊天内容。本地排错先学会按顺序查看三处：浏览器 Network，后端日志，SQL 或模型响应。

## 5.6 认证与授权

认证确认身份，英文是 Authentication，回答“你是谁”，例如用邮箱和密码登录。授权确认能做什么，英文是 Authorization，回答“你能做什么”，例如只能读取自己的会话。

身份，Token 与资源归属是一条连续的链路：

1. 注册时提交邮箱，密码和昵称。
2. 密码经 Argon2 计算，结果保存为 `password_hash`。
3. 用户记录写入 `users` 表。
4. 登录时后端校验邮箱与密码。
5. 校验通过后签发 JWT。
6. 之后每次访问资源，后端都要做资源授权。

Token 证明用户的身份，但对每一个 `conversation_id`，后端仍需执行资源归属校验。

## 5.7 注册与登录接口

注册接口是 `POST /api/v1/auth/register`，接收邮箱，密码和昵称。后端校验格式与密码强度，并把邮箱规范化。数据库的“唯一约束”作为最后一道检查，防止并发注册时写入重复的邮箱。注册成功返回 `201` 和安全的用户信息，邮箱重复时返回 `409`。

密码不能明文保存。密码使用专门的“慢哈希”算法加盐后保存，课程采用 Argon2，数据库里只存计算结果 `password_hash`。加盐是在计算之前给每个密码混入一段随机数据，相同的密码因此得到不同的结果。登录时比较的是计算结果，不需要也不能“解密回原密码”。普通的 SHA-256 速度太快，不适合直接作为密码存储方案。数据库一旦泄露，攻击者可以用这类快速算法高速尝试大量候选密码，Argon2 这类慢速算法则故意让每次计算变慢，提高逐个猜测的成本。

登录接口是 `POST /api/v1/auth/login`，校验账号状态，并把密码与 `password_hash` 比对。校验成功时签发有过期时间的访问 Token。校验失败时统一返回 `401`，避免泄露账号是否存在。登录请求以明文传输密码，所以生产环境必须使用 HTTPS，本地只在 localhost 演示。

完成后要能提供三项证据：

- 错误密码返回 `401`。
- 过期或伪造的 Token 被拒绝。
- 跨用户访问资源时返回 `403` 或 `404`。

## 5.8 JWT 与身份携带

JWT 由三部分组成：Header 写签名算法 `alg` 和 Token 类型 `typ`，Payload 写用户标识 `sub` 和过期时间 `exp` 等声明，Signature 是用 HMAC 或 RSA 算出的签名。JWT 的载荷可以被读取，可信性来自签名与服务端校验，所以 Payload 中不能放密码，API Key 或敏感资料。

服务器用密钥验证签名，确认 Token 没有被篡改，再检查 `exp` 与签发方等声明，最后校验请求的资源是否归属当前用户。`sub` 放稳定的用户标识，Token 必须设置过期时间，Token 中不保存密码。

前端用 Bearer 方案携带身份，请求头写成 `Authorization: Bearer <token>`。学习阶段可以先用容易理解的方案完成整条链路，同时要清楚 Token 存在 `localStorage` 中面临 XSS 风险：页面一旦被注入恶意脚本，脚本就能读取其中的 Token。生产环境常采用更完整的策略，例如短期 Access Token 配合 Refresh Token，或者使用 HttpOnly 与 Secure Cookie，HttpOnly 的 Cookie 不能被页面脚本读取。

`GET /api/v1/users/me` 是当前用户接口，从已验证的 Token 中获取用户身份，忽略客户端自报的 `user_id`。前端启动时调用这个接口，恢复登录状态与用户资料。Token 无效或过期时接口返回 `401`，前端清理状态并跳转到登录页。

## 5.9 对象级权限与状态码

**对象级权限**检查的是资源归属：知道 `conversation_id` 不代表有权访问这条会话。AI 生成的代码常常只加 JWT 中间件，没有校验对象归属，这是容易漏掉的漏洞。

危险写法只按 id 查询，例如 `SELECT * FROM conversations WHERE id=?`，登录成功的用户可以逐个尝试 id，读到别人的资源。正确的边界是同时按 id 与当前用户查询，条件写成 `WHERE id=? AND user_id=?`。读与写两类操作都执行同一规则：读取历史属于读，上传，更新和删除属于写。对外可以返回 `404`，减少资源存在性的泄露。

验收方法：用 A 用户创建会话，再用 B 用户的 Token 访问，访问必须失败，并且没有数据泄露。

状态码按身份与权限的情况选择。未认证用 `401`，已认证但没有角色权限通常用 `403`，访问其他用户的私有资源可以返回 `404`，避免暴露资源确实存在。课程项目要保持同一种策略，并写入接口契约。

## 5.10 CORS 与认证模块验收

CORS 源于浏览器的同源策略。同源策略按协议，主机，端口判断 Origin，三项中任何一项不同就是跨源。Day 2 的前端运行在 `localhost:5173`，调用运行在 `localhost:8000` 的 Day 3 后端，端口不同，属于跨源请求。浏览器对跨源请求做安全检查，后端没有声明允许这个来源时，前端页面读不到响应。处理方式是后端明确配置允许的本地前端 Origin，方法和 Headers，前端保持正常的 CORS 请求模式。

CORS 有两种错误做法：带 Cookie 或凭据时简单使用任意来源 `*`，以及为了“消除报错”把所有域名，方法和 Headers 永久开放。另外要明确，CORS 约束的是浏览器的跨源读取，API 的身份认证和授权需要独立实现。

认证模块按三组结果验收：

- 注册成功，重复邮箱返回冲突，强度不足或格式非法的输入被拒绝。
- 正确密码登录成功，错误密码返回 `401`，过期的 Token 被拒绝。
- A 用户不能访问 B 用户的会话，即使知道了会话 ID。

## 5.11 CRUD 接口清单

会话的基础接口有三个：

| 方法 | 路径 | 作用 |
| --- | --- | --- |
| `POST` | `/api/v1/conversations` | 创建会话 |
| `GET` | `/api/v1/conversations` | 查询当前用户的会话列表并分页 |
| `GET` | `/api/v1/conversations/{id}` | 返回会话及消息 |

`PATCH` 和 `DELETE` 可以作为提高项。所有接口都必须校验归属。

---

**小结**

FastAPI 项目从一个能启动，能访问，能调试的最小工程起步，目录按职责分层，配置集中在环境变量里。请求处理有三条约定：每个请求使用独立的数据库 Session，错误用稳定的结构返回，接口文档随代码自动生成。认证解决“是谁”，授权解决“能做什么”，密码只以 Argon2 加盐计算后的结果保存，JWT 只携带可以公开的声明。登录之后，每一次读写会话仍要确认资源归属，CORS 配置只决定浏览器能否读到响应，不承担权限检查。

# 06
# 对话 API 及模型接入与持久化

会话 CRUD 完成之后，接入模型，并把完整的对话历史保存下来。

## 6.1 发送消息主链路

发送消息 API 是整套系统的主链路。一次消息从校验开始，到持久化结束，后端依次完成六步：

1. 校验登录状态与会话归属，检查内容长度，并建立幂等上下文。
2. 保存 `user` 消息。
3. 构造发给模型的上下文。
4. 调用模型。
5. 保存 `assistant` 消息，更新会话时间。
6. 把结果返回前端，或者以流式输出推送给前端。

任何一步失败时，后端记录可以恢复的状态，并返回对应的错误码和重试信息。用户消息在调用模型之前已经落库，所以模型失败时，数据库里仍有真实的请求记录。

## 6.2 API Key 与模型 Client

模型的 API Key 只能由后端保管。浏览器中的源码，Network 记录和打包产物都能被用户查看，前端直连模型会暴露密钥，也无法在一处统一实现限流与重试，审计与成本控制。正确的链路是浏览器调用自己的后端，后端再调用模型供应商。

后端通过兼容式的模型 Client 与供应商隔离。兼容式指多家供应商提供相同调用格式的接口，同一套代码只要更换 `base_url`，`api_key` 和 `model` 三项配置，就能切换到另一家供应商。这三项写在环境配置里，供应商 SDK 由独立的 `ModelClient` 封装，模型调用代码集中在这里，不分散写在各个路由中。DeepSeek，GLM 等具体模型的能力会变化，项目模板要能通过修改配置替换模型。

## 6.3 消息角色与模型参数

Chat 消息用 `role` 区分来源：`system` 写应用规则和角色边界，`user` 是用户输入，`assistant` 是模型的历史回复。模型不会自动记住之前的对话，多轮对话必须在每次请求中按顺序带上必要的历史，否则模型无从参考前面的对话内容，表现为“失忆”。数据库保存完整历史，发送给模型的上下文则根据窗口与成本选取必要内容，窗口指模型一次能接收的上下文长度上限。

模型的常用参数有四个：

| 参数 | 作用 |
| --- | --- |
| 模型名 | 选择具体使用的模型 |
| 最大输出 | 限制一次回复的长度 |
| 温度 | 控制输出的随机性 |
| 停止条件 | 生成到指定内容时结束 |

温度没有适用于所有场景的“最佳值”，客服场景更看重稳定，可控和可追踪。参数名和可用范围以供应商的当前文档为准，不让 AI 凭印象编造。

## 6.4 消息保存与事务边界

发送消息时先保存用户消息，再调用模型。推荐的做法是保存 `user` 消息并标记状态，然后调用模型，成功后保存 `assistant` 消息。调用失败时，把这一轮标记为 `failed`，用户可以重试，输入也不会丢失。

这个顺序与事务边界有关。数据库事务适合保护一组快速的本地数据变更。外部模型调用慢，而且可能超时，不应把数据库锁一直带到网络调用结束。所以模型调用之前要先结束数据库事务，避免长时间占用连接。生产环境可以用任务队列或事件进一步拆分处理流程，本地先实现清晰的状态机。

完成后要能提供三项证据：

- 用户消息先写入数据库。
- 模型调用有超时与失败分支。
- 助手消息落库后可以刷新恢复。

## 6.5 流式输出与事件协议

非流式输出等完整回复生成后一次返回，实现简单，但用户等到第一个字出现的时间长。流式输出在模型产生 Token 时逐步推送，用户更早看到内容，所以体验更好。浏览器端可以使用 Fetch 流或 SSE。WebSocket 适合真正需要双向实时通信的场景，聊天回复只需要服务器单向推送，不必为此引入 WebSocket。

流式接口需要明确的**事件协议**，事件类型约定为四种：

| 事件 | 含义 |
| --- | --- |
| `start` | 一次回复开始 |
| `delta` | 携带新生成的一小段内容，可以连续出现多次 |
| `done` | 回复正常结束，附带 message ID 与用量等最终元数据 |
| `error` | 回复因错误结束 |

每个 chunk 必须有明确的边界。网络分块不等于完整的 JSON，不能假设一次网络读取得到的就是一个完整的 JSON。连接中途断开的情况也要能被识别。

## 6.6 超时重试与限流

模型调用必须设置连接超时和总超时，不能让请求无限挂起。重试只针对适合重试的临时错误，并设置次数上限，每次重试前的等待时间按**指数退避**成倍增加。用户重复点击与后端自动重试都可能产生重复消息，所以需要请求 ID 与幂等的思维，让重复到达的同一个请求只生效一次。

为了控制成本和保护资源，后端按用户或 IP 限制请求频率与并发数，同时限制输入长度与历史轮数。每次模型调用都记录所用模型与 Token 用量，以及耗时和结果状态，便于成本核算与排错。429 表示请求频率超过限制，此时要给前端可以理解的提示，并采用有上限的退避重试。

## 6.7 输出校验与对话标题

模型输出需要校验边界。模型可能产生幻觉，生成看似合理但与事实不符的内容，也可能遗漏信息或生成不合规的内容。业务关键数据必须来自数据库，API 或 RAG，不依赖模型训练时记住的知识。展示模型输出时，不能把模型生成的 HTML 或 Markdown 当作可信代码直接执行。

对话标题有两种生成策略。简单的做法是截取首条用户消息作为标题，不产生额外的模型成本。提高项是异步让模型总结一个短标题，生成失败也不影响聊天主流程。两种做法生成的标题都要保存到数据库，不能每次刷新都重新随机生成。

## 6.8 上下文长度与对话验收

历史上下文过长时，先按最近 N 轮或 Token 预算做限制，不能把数据库里的所有消息无限制地发给模型。早期对话可以用摘要压缩，原始消息仍然保存在数据库。Day 4 的 RAG 负责检索外部知识，检索结果在调用模型之前加入上下文，对应发送消息链路中构造上下文的一步。聊天历史的压缩与管理则需要单独设计。

对话功能按三项证据验收：

- 浏览器 Network 中能看到真实的后端请求，后端日志能看到模型调用。
- MySQL 中同时出现 `user` 和 `assistant` 消息，状态与顺序都正确。
- 刷新页面，退出再登录，重启后端之后，历史仍然可以从数据库加载。

---

**小结**

发送消息的核心顺序是先落库用户消息，再调用模型，最后落库助手回复，任何一步失败都留下可以恢复的状态。模型密钥只在后端出现，供应商之间的差异由配置与独立的 Client 隔离。数据库事务只覆盖快速的本地写入，耗时的模型调用放在事务之外，并配上超时，有限重试与限流。流式输出依靠事件协议划清每一段内容的边界，上下文按窗口与成本裁剪，完整历史始终以数据库为准。

# 07
# 第二天前端接上第三天后端

联调的顺序是先验证接口，再接页面，联调中出现的问题逐层定位。

## 7.1 前端改造范围

Day 2 的页面保留，改的是数据来源，由假数据改为真实 API。前端的改造集中在 API，身份和会话状态三方面，按链路顺序涉及六个部分：

| 改造部分 | 改动内容 |
| --- | --- |
| 环境变量 | 用公开变量保存后端的 Base URL |
| API Client | 集中管理 Base URL，认证和错误处理 |
| Token 状态 | 处理登录与退出，恢复当前用户，处理 Token 过期 |
| 会话 Store | 会话列表与消息列表改为读取后端的历史接口 |
| 消息状态机 | 发送按钮调用真实的对话接口，处理 loading，stream，error 三种状态 |
| 错误提示 | 按统一的错误对象提示用户 |

## 7.2 前端环境变量与 API Client

后端的 Base URL 用框架约定的公开变量保存，本地开发时的值例如 `http://localhost:8000/api/v1`。公开变量会打包进浏览器端的代码，所以只能放可以公开的配置，模型 API Key 和数据库密码不能放进去。生产地址由构建或部署环境注入，不能在各个组件里直接写 localhost。

统一的 API Client 负责五件事：

- 拼接 Base URL，完成 JSON 序列化，附加 Authorization Header。
- 把状态码不在 2xx 范围内的响应转换为统一的错误对象。
- 收到 `401` 时触发登录失效流程。
- 支持请求取消与超时。
- 普通 JSON 请求与流式响应分别处理，不使用同一种解析方式。

## 7.3 登录注册页面联调

登录注册页面联调时，表单的前端校验用来提升体验，后端校验负责安全，两层校验都要有。页面要分别显示重复邮箱，密码错误和网络失败等不同状态。登录成功后调用 `/users/me`，再次验证 Token 并加载当前用户。

会话列表联调时，页面加载后查询当前用户的会话，并按更新时间展示。新建会话后，用后端返回的真实 ID 更新路由和状态。

## 7.4 发送消息的状态机

发送消息的界面按状态机流转：从空闲状态 `idle` 进入发送中或接收流式内容的 `sending/streaming`，最后到达成功的 `success`，或者出错与取消的 `error/cancelled`。发送期间要避免重复提交，同时支持停止生成或重试失败的消息。页面先显示的临时消息，在完成后与数据库中真实的 message ID 对齐。

完成后要能提供三项证据：

- 在 Network 中核对请求参数。
- 用后端日志定位业务分支。
- 用数据库查询确认最终状态。

## 7.5 浏览器里的联调证据

浏览器按 F12 打开开发者工具，其中的记录是前后端联调的证据，重点看四处：

- Network：请求是否发出，URL 和方法是否正确，Payload 是否匹配。
- Headers：Origin，Content-Type 与 Authorization 是否存在。
- Response 或 Preview：状态码，错误结构和流式内容。
- Console：只作补充，不能把所有 4xx 和 5xx 错误都误判为 CORS 问题。

联调检查按固定顺序进行：

```
Network -> Request URL -> Method / Status ->
Request Headers -> Request Payload -> Response Body
Console -> CORS / JS 异常
后端日志 -> 路由 / SQL / 上游模型
```

顺序是先在 Network 中核对请求与响应，再到 Console 查看 CORS 与 JS 异常，最后在后端日志中查路由，SQL 与上游模型。

### 7.6 三层测试法

接口测试分三层：curl 验证网络与后端的底层行为，Swagger 验证接口流程与 Schema，前端验证用户体验，状态管理和跨域。如果 Swagger 测试成功而前端失败，说明后端接口可用，优先检查前端的请求构造与 CORS。

一类典型错误是前端显示成功，后端实际失败，常见原因有三种：

- 没有检查 HTTP 状态码，收到错误的 JSON 仍然追加一条“成功”消息气泡。
- **乐观 UI** 在请求返回之前先显示成功的结果，失败时没有回滚，数据库里实际没有记录。
- 用假的 fallback 回复掩盖模型错误，演示看起来顺畅，系统却无法审计。

### 7.7 完整演示流程

前后端接通后，按固定顺序演示整条链路，某一步失败时可以定位到具体环节。演示分四段：

1. 新用户注册并登录，创建会话后发送问题，看到流式回答。
2. 打开数据库客户端，展示用户，会话和两条消息记录，其中一条是 `user` 消息，一条是 `assistant` 消息。
3. 刷新页面并重新登录，再重启后端，然后再次打开历史会话。
4. 用第二个账号修改 URL 或会话 ID，证明无法读取第一个账号的数据。

---

**小结**

前端改造的原则是页面保持不变，数据来源换成真实 API，改动集中在请求封装，身份状态与消息状态三处。只有可以公开的配置才放进前端变量，所有请求经过统一的 API Client，普通 JSON 与流式响应分开解析。联调从底层往上逐层排查：curl 与 Swagger 确认接口，浏览器开发者工具确认请求与响应，后端日志与数据库确认最终结果。

# 08
# AI/Vibe Coding 常见工程问题与排错

AI 可以快速生成代码，代码是否真的完成，由工程证据决定。

### 8.1 假后端与假完成

AI 生成的后端里，常见的一类假后端是用内存数组冒充数据库。页面能动，接口返回 200，都不能证明数据真的持久化了。看起来完成，实际没有完成的情况统称**假完成**，每一种都可以用一步操作识别：

| 假完成 | 表面现象 | 识别方法 |
| --- | --- | --- |
| 内存数组 | 增删改看似正常 | 重启后端后再次查询 |
| localStorage | 刷新仍能看到 | 换浏览器或清空站点数据 |
| Mock 或 setTimeout | 聊天有回复 | 查看 Network 是否调用真实模型链路 |
| 假登录 | 页面显示用户名 | 用错误密码，过期 Token 和跨用户资源测试 |
| 假数据库 | GUI 里有表 | 查看本次请求对应的 SQL 与真实新增的记录 |

内存数组里的数据随后端进程重启而消失，localStorage 只存在于当前浏览器，所以重启后端和更换浏览器就能暴露这两种假完成。判断的原则是：任何“完成”都要经得起两类检验，Network 与日志证明请求真实发生，数据库查询与重启测试证明数据真实保存。

## 8.2 假登录与密钥泄露

AI 写出的“假登录”常见三种：

- 任意密码都能进入，前端把邮箱写进状态就视为登录。
- 后端签发了 Token，受保护的接口却没有校验。
- 有认证没有授权，登录后可以读取所有用户的会话。

密钥泄露同样常见，泄露途径有三类：写进前端或源码，出现在 README 与截图中，留在日志或 Git 历史里。`.gitignore` 是最后一道基本防线，只对还没有被 Git 跟踪的文件起作用，已经提交的 Key 仍然留在历史记录里，不会因此消失。Key 一旦泄露，要立即到供应商后台吊销或轮换，作废旧 Key 并换用新 Key，只删除本地文件不够。

## 8.3 编造的依赖与过大的改动

AI 会编造包，方法和过期教程，表现为包名相似但实际不存在，API 已经改名，不同大版本的代码混在一起。应对的办法是让 AI 先读取当前的依赖清单与官方文档，依赖清单指 `pyproject.toml`，`package.json` 或 `pom.xml`，再用实际的安装结果，类型检查和测试输出判断代码是否可用。

AI 还容易一次修改太多文件。前端与后端，数据库与配置同时大范围修改，出错后无法定位问题。改法是按**垂直切片**推进，每次只完成一个从接口到数据库都能运行的小功能：先完成 health 检查与 DB 连接，再完成注册，登录与会话，最后接入消息与模型并整体验收。每一步通过小验收并提交 Git，再进入下一步。

## 8.4 异常处理与同步异步混用

AI 生成的代码容易吞掉异常，即捕获异常之后既不记录也不上报。例如这样的写法会把真实故障伪装成成功：

```
except Exception: return {"success": True}
```

返回给用户的响应要安全，简洁，服务端日志则要保留可定位的错误类别与堆栈。空的 catch，没有日志的 fallback，永远返回 200，这三种写法都不允许。

同步与异步代码混用也是常见问题。`async` 并不会自动让阻塞的数据库驱动或 SDK 变成非阻塞，阻塞调用会拖慢并发。学习阶段可以先选一致，简单的方案，生产环境再按测量结果优化。不让 AI 为了“高性能”同时引入异步 ORM，线程池和队列，增加不可控的复杂度。

## 8.5 数据库与接口故障

数据库的常见故障可以分成五类：

| 类别 | 常见故障 |
| --- | --- |
| 服务与地址 | Server 未启动，端口不对 |
| 账号与数据库 | 账号无权限，数据库不存在 |
| 连接配置 | URL 中密码的特殊字符未编码，字符集不一致 |
| 表结构 | 迁移没有执行 |
| 连接资源 | 连接池耗尽，Session 未关闭 |

排查时先读最早出现的根因异常，后面的报错往往是连带结果。接口的常见故障按状态码区分：

| 状态码 | 常见原因与处理 |
| --- | --- |
| 404 | 路径或 /api/v1 前缀错误 |
| 405 | HTTP 方法错误 |
| 422 | JSON 的字段或类型与 Schema 不匹配 |
| 401 | Token Header 缺失或过期 |
| 500 | 后端出错，先去后端日志查根因，再让 AI 修改对应的服务代码 |

## 8.6 流式故障与重复扣费

流式响应的常见故障有五种：

- 前端把流式响应当作 `response.json()` 处理。
- 中文被错误地分块解码。一个中文字符由多个字节组成，网络分块可能把一个字切成两半，所以解码时不能把每一块单独转成文字。
- 代理或服务器缓冲了响应，内容最后一次性出现。

- 连接中断时没有发送结束事件。
- 数据库只保存了部分回复，需要明确中断状态与最终提交的时机。

重复消息与重复扣费来自多次模型调用，用户连续点击，网络重试和后端自动重试都可能触发。前端禁用重复提交只是体验上的措施，后端还需要请求 ID 与幂等检查的思维。后端记录模型请求的状态，结果未知的请求不能简单重放。

## 8.7 提示词的结构

高质量提示词先锁定边界，步骤与验收，再让 AI 修改代码。写给 AI 的要求包括四点：

- 先读现有的目录与依赖，接口与 Day 2 代码，保留已经完成的 UI。
- 先输出最小变更计划与验收命令，再按单一垂直切片实施。
- 明确技术栈与版本，接口 Schema 与表结构，安全边界与禁止项。
- 完成后运行格式化，类型检查或测试，并报告实际输出和未完成项。

提示词按六项书写：

```
角色：你是资深后端工程师
任务：实现一个垂直切片
约束：技术栈、目录、接口契约、安全边界
步骤：先计划，再逐文件修改
验收：curl / 测试 / SQL 证据
输出：变更摘要与风险
```

验收一项写明 curl，测试和 SQL 证据，AI 交回结果后就能逐项核对。

## 8.8 AI 排错所需材料

用 AI 排查 Bug 时，需要提供三类材料：

- 复现步骤，预期行为与实际行为，完整的首个错误和相关日志。
- 操作系统与运行时版本，启动命令与相关依赖的版本。
- Network 的请求与响应，以及最小的相关代码，其中的密码，Token 与 Key 要隐去。

收到材料后，先让 AI 提出根因假设和验证方法，再按证据修改。完成后要能提供三项证据：

- 给出可以稳定复现的步骤。
- 提供完整的报错与环境信息。
- 只修改最小范围，并重新验证。

**小结**

判断 AI 写的功能是否完成，依据是工程证据：请求要在 Network 与日志里留下记录，数据要在数据库查询和重启之后依然存在。认证，授权与密钥是假完成的高发区，泄露的 Key 必须到供应商后台作废。控制 AI 的方法是限定改动范围，先核对真实的依赖与文档，再按垂直切片小步推进，每一步验收后提交。出了问题按状态码与根因异常定位，把可复现的步骤与完整报错交给 AI，再按证据修改。

# 09
# 课后作业与评分

作业验收围绕三件事：真实持久化，真实身份，真实的模型调用链路。

## 9.1 作业要求与真实持久化

课后作业的目标是让智能客服真正可以对话。作业在 Day 2 前端的基础上接入自己的后端和 MySQL，必须使用真实的模型 API，模型 Key 只能放在后端的环境变量中。功能上必须实现注册与登录两项账号功能，以及新建会话，发送消息，读取历史三项会话功能。

基础必做验收分三组。第一组是真实持久化：发送一轮问题后，数据库中能查到 `user` 与 `assistant` 两条消息，刷新页面，关闭浏览器，重启后端之后，重新登录仍能看到历史。主存储必须是 MySQL，内存与 localStorage，JSON 文件与前端 mock，都不能代替 MySQL 作为主存储。

## 9.2 身份隔离与接口质量

第二组是真实身份与隔离。数据库只保存密码的 `password_hash`，错误密码不能登录，重复邮箱返回正确的错误码 `409`。A，B 两个账号的数据相互隔离，修改会话 ID 不能越权读取。模型 Key，数据库密码与 JWT Secret 不出现在 Git 与前端产物中。

第三组是真实接口质量。API 的路径与方法，输入输出与状态码设计合理，Swagger 可用。前端区分加载中与空数据，成功与失败几种状态，不把错误显示成成功。按 README 能在新环境中完成安装与迁移，启动与验证。

## 9.3 提高项与提交材料

基础功能达到要求之后，提高项用来区分完成质量，分四组：

- 流式输出，停止生成，失败重试。
- 限流与超时，幂等请求与请求 ID，更完整的故障恢复。
- 会话的重命名与删除，分页与自动标题，Token 与成本统计。
- 最小自动化测试和更完整的日志。

提交材料有四项：

- Git 仓库，前端代码放在 `frontend/` 目录，后端代码放在 `backend/` 目录，仓库里还要有迁移文件，README 和 `.env.example`。
- 两到四分钟的录屏，先演示注册，登录与对话，再查数据库并演示刷新或重启后恢复，最后演示双用户隔离。
- 接口的截图或导出文件，数据库表结构与关键记录的截图。
- 一页项目说明，写明架构与技术栈，难度最高的问题与排错证据，以及下一步计划。

## 9.4 评分与返工项

评分只看能否提供工程证据，分值按六个维度分配：

| 维度 | 可验证证据 | 分值 |
| --- | --- | --- |
| “基础闭环” | 注册登录，会话 CRUD，真实模型，刷新恢复 | 35 |
| 数据库 | 表设计，约束，迁移，应用账号，真实记录 | 20 |
| 接口质量 | OpenAPI，状态码，结构化错误，CORS，安全边界 | 15 |
| 工程质量 | 分层，配置，README，锁文件，Git 提交 | 15 |
| 流式与可靠性 | 事件协议，中断处理，超时重试，限流与幂等 | 10 |
| 展示表达 | 演示顺序，故障复盘，架构与取舍说明 | 5 |

六个维度合计 100 分。一票否决的情况有五种，出现任何一种，基础项都必须返工重做：

- 密码明文存储。
- 模型 Key 出现在前端或仓库。
- 用假回复冒充真实模型，或者用内存与 localStorage 冒充数据库。
- 登录后可以读取其他用户的会话。
- 项目无法按 README 启动，并且没有说明依赖和环境。

验收时要提供三项证据：功能可以操作并覆盖失败分支，数据可以查询并在重启后仍然存在，身份隔离与密钥管理可以证明。

## 9.5 理解程度自检

这一天的学习效果用十个问题检验，问题围绕三条线索展开：数据怎样持久化，身份与密钥怎样保护，一条消息怎样在前后端，数据库与模型之间流转。能完整回答这些问题，才说明已经掌握后端，数据与模型链路。回答时要说清边界与数据流，失败路径与验证证据。

**小结**

这份作业检验的是一条真实可用的全栈链路：数据真正落库，身份真正隔离，模型真正被调用。基础必做项决定能否通过，提高项决定完成质量，每一项得分都要有可以复查的证据。密钥外泄与明文密码，跨用户越权，假持久化属于原则问题，出现任何一项都要返工。

# 10
# 环境安装命令附录

附录汇总可以直接执行的安装命令，版本检查与连通性自检。

## 10.1 Windows 安装前检查

Windows 环境统一使用 64 位的 Windows 10 或 11。命令要区分 PowerShell 与 CMD，复制命令时去掉 `$` 或 `PS>` 提示符。安装前先在 PowerShell 中检查旧环境：

```
Get-Command uv,python,mysql `
-ErrorAction SilentlyContinue
where.exe uv
where.exe python
where.exe mysql
```

`Get-Command` 一次检查三个命令是否存在，第一行末尾的反引号是 PowerShell 的续行符，两行合起来是一条命令。系统中已经装有多个版本时，先用 `where.exe` 记录 uv，python 与 mysql 的实际位置，再有依据地调整 PATH。安装完成后必须关闭并重新打开终端，必要时重启 VS Code，让新的 PATH 生效。

## 10.2 Windows 安装 uv 与 Python

Windows 上安装 uv 有两种方式。官方脚本的命令是：

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

也可以用 winget 安装。用 winget 安装 uv 并准备 Python 3.12 的完整命令是：

```
winget install --id=astral-sh.uv -e
uv --version
uv python install 3.12
uv python pin 3.12
uv run python --version
```

安装完成后在新终端执行 `uv --version`，再用 `uv python install 3.12` 安装 Python 3.12，用 `uv python list` 查看已安装的版本。进入项目后执行 `uv python pin 3.12` 固定版本，再用 `uv run python --version` 确认项目实际使用的解释器。这种做法不依赖系统的 `python` 命令，即使系统的 `python` 命令指向应用商店的占位符，也没有影响。

企业电脑无法执行脚本时，使用官方发布的可执行文件或 winget 安装，并请管理员处理策略，不建议永久关闭安全策略。

## 10.3 macOS 与 Linux 安装 uv

macOS 与 Linux 用官方脚本安装 uv，macOS 也可以用 `brew install uv`。安装后根据提示重新打开终端，或者加载 shell 配置，再执行 `uv --version`。之后依次安装，固定并确认 Python 3.12，完整命令是：

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv --version
uv python install 3.12
uv python pin 3.12
uv run python --version
```

Apple Silicon 与 Intel 芯片的 Mac 上，可执行文件的路径可能不同。排错时用 `which uv` 查看 uv 的实际位置，用 `uname -m` 查看机器架构。

## 10.4 Windows 安装 MySQL

项目使用 MySQL 8.4 LTS，LTS 是长期支持版本。Windows 上的安装步骤是：

1. 下载官方的 MySQL 8.4 Windows x64 MSI 安装包。
2. 用 MySQL Configurator 创建 Windows Service。
3. 选择开发机配置，TCP 端口 3306，设置强 root 密码。
4. 记录服务名，设置随系统启动。

安装后的检查命令依次是：

```
Get-Service *mysql*
mysql --version
mysql -u root -p
SELECT VERSION();
SHOW VARIABLES LIKE 'port';
```

前三行在 PowerShell 中执行，`Get-Service *mysql*` 检查服务状态，`mysql -u root -p` 输入密码登录。登录之后，后两行 SQL 在 mysql 客户端里执行，分别查看 Server 版本和端口。如果 `mysql` 命令不存在但服务正常，把 MySQL 的 `bin` 目录加入 PATH，或者先用完整路径或 Workbench 连接。

## 10.5 macOS 安装 MySQL

macOS 上先在官方 DMG 或 PKG 安装包与 Homebrew 之间选定一种安装方式，并使用对应的服务管理命令。官方包在系统设置的 MySQL 面板中管理服务，Homebrew 安装则用 `brew services` 管理。

`mysql --version` 只说明客户端存在，执行之后仍要用 `mysql -u root -p` 登录，并运行 `SELECT VERSION;` 验证 Server。验证命令是：

```
mysql --version
mysql -u root -p
SELECT VERSION();
# Homebrew 方案示例
brew services list
```

其中 `brew services list` 只适用于 Homebrew 安装，用来查看服务状态。Apple Silicon 的 Mac 找不到客户端时，检查 `/usr/local/mysql/bin` 或 Homebrew 前缀，只把实际存在的目录添加到 PATH。

## 10.6 课程数据库与连通性验证

课程数据库的创建与验证分四步：

1. 用 root 登录，创建 `customer_service` 数据库，字符集选择 `utf8mb4`，再创建仅用于本项目的本地应用账号。
2. 授权范围只限 `customer_service.*` 这一个数据库，然后退出 root，用应用账号重新登录。
3. 依次执行 `SELECT VERSION();`，`SHOW DATABASES;`，`USE customer_service;`，证明版本，身份和权限都正确。
4. 让 FastAPI 执行一个数据库健康检查。

建库，建账号与授权的 SQL 是：

```
CREATE DATABASE customer_service CHARACTER SET utf8mb4;
CREATE USER 'customer_app'@'localhost' IDENTIFIED BY '***';
GRANT ALL ON customer_service.* TO 'customer_app'@'localhost';
SHOW GRANTS FOR 'customer_app'@'localhost';
```

应用不使用 root 连接数据库。root 只用于数据库管理，应用账号只获得这个项目需要的权限，账号一旦泄露，影响范围也只限这一个数据库。代码中的应用账号以 `customer_app` 为例，`'***'` 处填写这个账号的密码，`@'localhost'` 表示这个账号只能从本机连接。`GRANT ALL ON customer_service.*` 只授予这一个数据库上的全部权限，`SHOW GRANTS` 核对授权结果。“GUI 能连”与“应用能用 DATABASE_URL 连”是两个验收项。

**小结**

安装之前先查清机器上已有的版本与路径，安装之后重开终端，再用版本命令确认结果。Python 由 uv 统一安装和固定，项目里通过 `uv run` 使用，不依赖系统自带的 `python`。MySQL 只看客户端版本不够，要登录 Server 执行查询。课程数据库配一个权限只限这一个库的应用账号，最后以 FastAPI 的数据库健康检查作为连通的证据。

# 练习与作业

## 复习题

回答时要说清边界与数据流，失败路径与验证证据。

1. 说明刷新页面后数据不丢失为什么不能只靠 React 或 Vue 的状态，并写出课后作业对真实持久化的验收要求。
2. 说明项目架构从 L0 到 L5 怎样逐级演进，并写出今天的智能客服至少要做到哪一级。
3. 写出一条聊天消息从浏览器出发再回到前端所经过的组件，以及发送消息接口在后端依次完成的步骤。
4. 说明 ORM 与数据库迁移分别解决什么问题。
5. 说明应用为什么不应使用 MySQL root 账号，并写出创建课程数据库和应用账号之后的验证步骤。
6. 比较 401，403 与 404 三个状态码的使用场合。
7. 说明 JWT 为什么不能保存密码，并写出后端校验 Token 时检查的内容。
8. 说明用户已经登录为什么仍可能发生越权，并写出防止越权的查询条件。
9. 说明模型 API Key 为什么不能放在前端，以及 Key 泄露后应该怎样处理。
10. 说明模型调用失败时数据库应该留下什么状态，以及先保存用户消息再调用模型的原因。
11. 说明 Day 4 的 RAG 将插入发送消息链路的哪一步，并比较 RAG 与聊天历史压缩各自处理的问题。
12. 比较 curl，Swagger 与前端三层测试各自验证什么，并写出浏览器开发者工具中联调检查的顺序。

## 课后作业

作业题目是让智能客服真正可对话。作业内容，基础必做验收，提高项，提交材料与评分规则都写在第 09 节，按 9.1 到 9.4 的要求完成并提交，提交前用 9.5 的自检问题检查一遍。